More Looping

CSL100

August 27

This Class

- Loops
  - For statement syntax, use and precautions
  - Using nested loops
  - Break and Continue statement

For loop in C

- General form

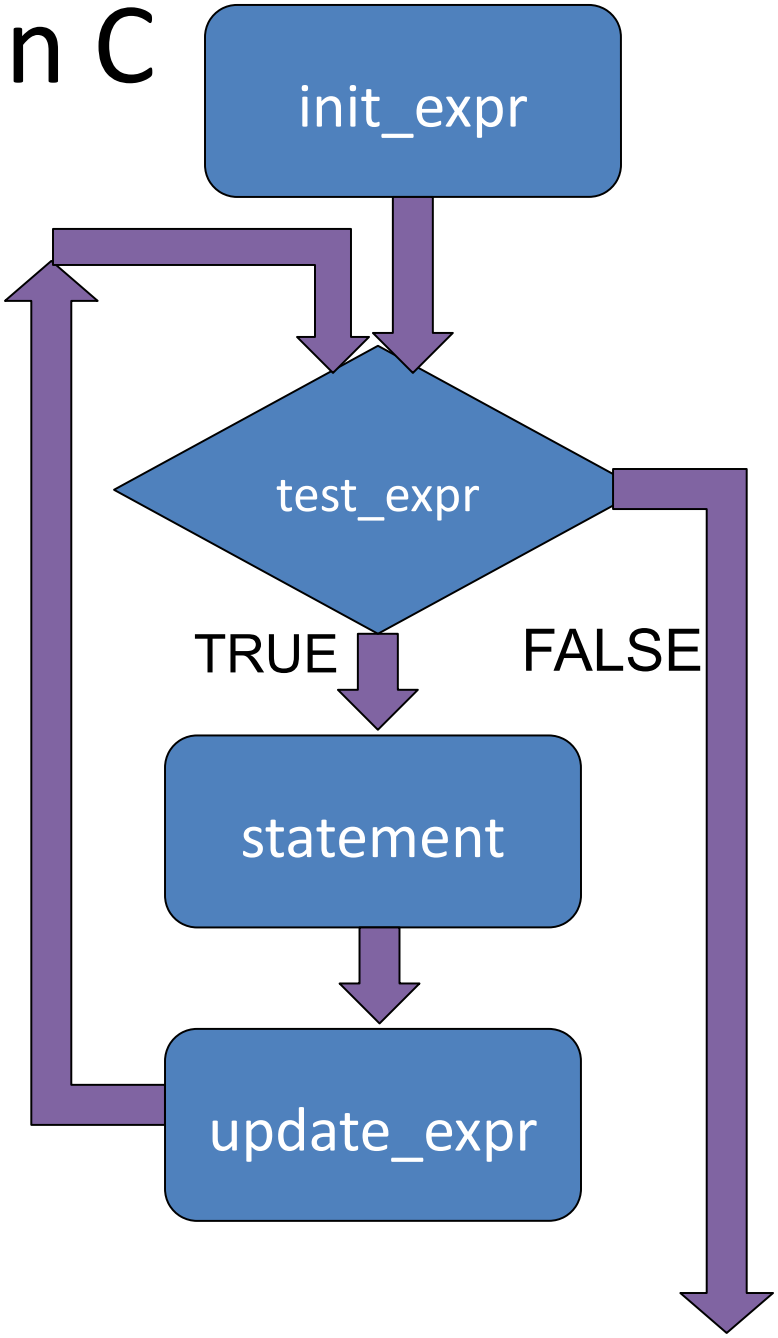
```
for (init_expr; test_expr; update_expr)
    statement;
```

- init_expr is the initialization expression.
- update_expr is the update expression.
- test_expr is the expression that evaluates to either TRUE (non-zero) or FALSE (zero).
- statement is the work to repeat (can be multiple statements in {…})

For loop in C

```
for (init_expr; test_expr; update_expr)
        statement;
```

1. First evaluate **init_expr**;
2. Evaluate **test_expr**;
3. If **test_expr** is TRUE then
   a) execute **statement**;
   b) execute **update_expr**;
   c) go to Step 2.
4. if **test_expr** is FALSE then break from the loop



For loop in Action

```
#include <stdio.h>
int main() {
    int i;
    for (i=1;i<=3; i++) {
        printf("%d\t",i)
    }
    printf("\n%d\t",i);

    return 0;
}
```

```
1 2 3
4
```

```
#include <stdio.h>
int main() {
    int i;
    for (i=1;i<=3; ++i) {
        printf("%d\t",i);
    }
    printf("\n%d\t",i);

    return 0;
}
```

```
1 2 3
4
```

Important Consideration: Scope

```
#include <stdio.h>
int main(){

 for (int i=1;i<=2;i++)
          printf("%d\n",i);
 return 0;
}
```

- Output?

1

2

Important consideration: Scope

```
#include <stdio.h>
int main(){

 for (int i=1;i<=2;i++)
          printf("%d\n",i);
 printf("%d\n",i);
 return 0;
}
```

- Output?

Compiler error: 'i' undeclared



Block Scope of a Variable

```
#include <stdio.h>
int main(){

 { //start block
 int i;
 for (i=1;i<=2;i++)
          printf("%d\n",i);

 } //end block

 return 0;
}
```

- Output?

  1

  2

Block Scope of a Variable

```
#include <stdio.h>
int main(){

 {
 int i;
 for (i=1;i<=2;i++)
          printf("%d\n",i);
 }
 printf("outside %d\n",i);

 return 0;
}
```

- Output?

Compiler error: 'i' undeclared

Block scope of a Variable

```
#include <stdio.h>
int main(){
 int i;
 for (i=1;i<=2;i++){
          printf("%d\n",i);

          int j=0;
          printf("j=%d\
n",j+1);
 }

 return 0;
}
```

- Output?

1

j=1

2

j=1

Flexibility in Expression Checking

```
#include <stdio.h>
int main() {
    int i, j = -1;
    for (i=1; j<=2; i++) {
            printf("%d\t",i);
            j = i;
    }
    printf("\n%d\t",i);
    printf("\n%d\t",j);
    return 0;
}
```

```
1 2 3
4 3
```

```
#include <stdio.h>
int main() {
    int i,j;
    for (i=1; j<=2;++i){
            printf("%d\t",i);
            j = i;
    }
    printf("\n%d\t",i);
    printf("\n%d\t",j);
    return 0;
}
```

```
1 gbg
```

Undesirable Flexibility

```
#include <stdio.h>
int main() {
    int i=1, j;
    for (;j<=2;i++){
        printf("%d\t",i);
        j = i;
    }
    printf("\n%d\t",i);
    printf("%d\n",j);
    return 0;
}
```

```
1  4200288
```

```
#include <stdio.h>
int main() {
    int i=1, j;
    for (;j<=2;i++){
        printf("%d\t",i);
        j = i;
    }
    printf("\n%d\t",i);
    printf("%d\n",j);
    return 0;
}
```

```
1    2    3
4    3
```

Nested Loops



- Loop with in a loop
- Many iterations of inner loop One iteration of outer loop





Example

- Write a program that displays the following pattern

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25
6	12	18	24	30
7	14	21	28	35
8	16	24	32	40

integers are printed in 5 columns each

```
#include<stdio.h>
int main(){
    int i, j;

    for (i=1; i<=8; i=i+1) {
        for (j=1; j<=5; j=j+1) {
            printf("%4d", i*j); // Displaying i, 2i, …, 5i
        }
        printf("\n"); // Move to the next line
    }

    return 0;
}
```

Displaying a Pattern

```
#include <stdio.h>
int main(){
  int i,j;
  for (i=1; i<=5; i=i+1){
    for (j=i; j<2*i; j=j+1){
      printf("%d ",j);
    }
    printf("\n");
  }
  return 0;
}
```

- Output?

1

2 3

3 4 5

4 5 6 7

5 6 7 8 9

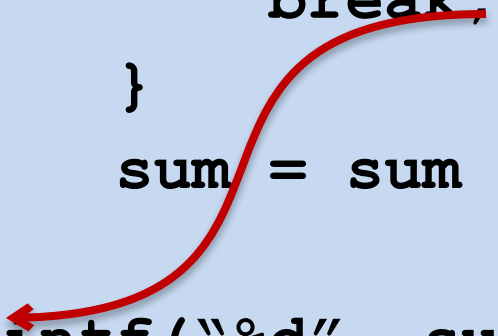
Back to Break



- Used for exiting a loop forcefully
- Example Program:

Read 100 integer inputs from a user. Print the sum of inputs until a negative input is found (Excluding the negative number) or all 100 inputs are exhausted.

```
int value;
int sum = 0;
int i;
for (i = 0; i < 100; i++) {
      scanf("%d", &value);
      if (value < 0) {
            //-ve number: no need to go
            // around the loop any more!!
            break;
      }
      sum = sum + value;
}
printf("%d", sum);
```



When to Break?

- Use of break sometimes can simplify exit condition from loop.
- However, it can make the code a bit harder to read and understand.
- Tip: if the loop terminates in at least two ways which are sufficiently different and requires substantially different processing then consider the use of termination via break for one of them.

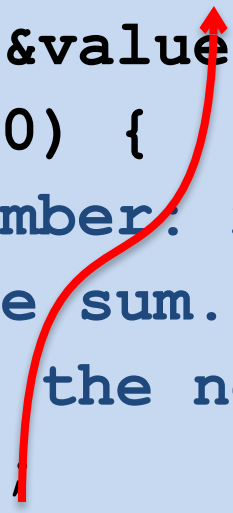
Continue

- Used for skipping an iteration of a loop
- The loop is NOT exited.
- Example Program:

Read 100 integer inputs from a user. Print the sum of only positive inputs.



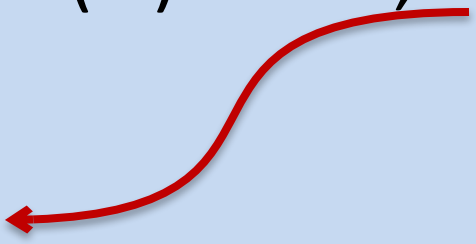
```
int sum = 0;
int i, value;
for (i = 0; i < 100; i++) {
     scanf("%d", &value);
     if (value < 0) {
          //-ve number: no need to add it
          // to the sum. Go ahead and
          // check the next input.
          continue;
     }
     sum = sum + value;
}
printf("%d", sum);
```



Break and Continue

- if there are nested loop: break and continue apply to the nearest enclosing loop only.

```
for (i = 0; i < 100; i++) {
   for (j = 0; j < 100; j++) {
     if (…) break;
   }
   …
}
```



Continue and Update Expr

- Make sure continue does not bypass update-expression for loops
  - Specially for while and do-while loops

```
i = 0;
while (i < 100) {
   scanf("%d", &value);
   if (value < 0) continue;
   sum = sum + value;
   i++;
}
```

i is never incremented
potentially infinite loop!!



Continue and Update Expr

- Correct Code:

```
i = 0;
while (i < 100) {
   scanf("%d", &value);
   if (value < 0) {
      i++;
      continue;
   }
   sum = sum + value;
   i++;
}
```

Continue and Update Expr

- Correct Code:

```
i = 0;
while (i < 100) {
   i++;
   scanf("%d", &value);
   if (value < 0) continue;
   sum = sum + value;
}
```

Big difference between *for* and *while*

```
#include <stdio.h>
int main() {
    int i = 0;
    for(i = 0; i < 10; i++) {
        if(i == 5) continue;
        printf("%d\n", i);
    }
    return 0;
}
```

```
0  1  2  3  4  6  7  8  9
```

```
#include <stdio.h>
int main() {
    int i = 0;
    while(i < 10) {
        if(i == 5) continue;
        printf("%d\t", i);
        i++;
    }
    return 0;
}
```

```
0  1  2  3  4  .....................
```

For loop in Terms of while Loop

```
for (init_expr; test_expr; update_expr)
        statement;
```

- Execution is (almost) equivalent to

```
init_expr;
while (test_expr) {
        statement;
        update_expr;
}
```

- Almost? Exception if there is a continue; inside statement
- Both are equivalent in power.
- Which loop structure to use?

Loop Best Practices

- I want to do something a times
  - for(i = 0; i < N; i++)

- I want to do something an indeterminate number of times until a condition is true
  - while (condition)

How Many Times is the Loop Executed?

```
int a = 10 - 6;
while (a < 10) {
        if (a = 5)  {
                printf("%d\n", a);
        }
        a=a+1;
}
```

Output
5
5
5
...



A common bug

Probable intention:

```
int a =10 - 6;
while (a < 10) {
        if (a == 5)  {
                printf("%d", a);
        }
        a=a+1;
}
```

Output
5

Next Class on Friday

- Introduction to functions (Ch 7)
  - Basic syntax
  - Basic use cases